

Dispositivos Conectados

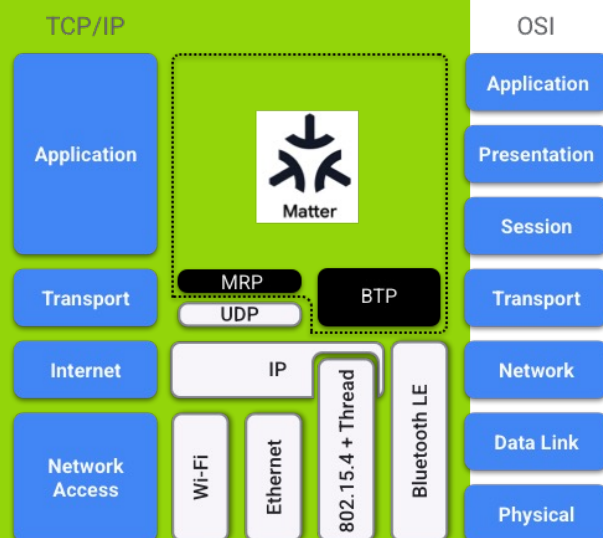
Protocolos IP para IoT: Modelo orientado ao recurso

Modelo de Comunicação Orientado ao Recurso



- Resposta à necessidade de **comunicação eficiente e padronizada** entre dispositivos interligados (frequentemente limitados em energia, memória, largura de banda, ...)
- **Modelo de comunicação baseado em recursos identificáveis**, geralmente representados através de URIs (*Uniform Resource Identifiers*).
 - Cada sensor, atuador ou serviço expõe as suas funcionalidades como recursos acessíveis que podem ser consultados, modificados ou observados remotamente.
 - Este paradigma deriva dos princípios do modelo REST (Representational State Transfer), amplamente utilizado na Web, adaptado a ambientes com restrições.
- As interações entre dispositivos seguem normalmente o **esquema cliente-servidor**, usando operações simples como GET, POST, PUT e DELETE.
 - Assim, um cliente pode, por exemplo, ler o valor de um sensor (GET), enviar um comando de controlo (POST) ou atualizar a configuração de um dispositivo (PUT).
- **Codificação e transporte da informação eficiente**: Em vez de formatos verbosos como JSON sobre HTTP/TCP, usam-se representações binárias compactas sobre UDP
- **Suporte a comunicação assíncrona**, através de mecanismos como Observe (em CoAP) ou Events (em Matter), que permitem a notificação automática de alterações de estado, aproximando-se de um modelo publish/subscribe.
- **Segurança integrada desde a origem**, com suporte a encriptação ponta-a-ponta (DTLS, OSCORE, TLS), autenticação mútua e gestão de identidades

Em que consiste o protocolo Matter?



2

- **Padrão interoperável** que visa promover a adoção tecnológica e a inovação, substituindo gradualmente protocolos proprietários dos ecossistemas de casas inteligentes.
- Implementado através de um **SDK de código aberto** que contém não apenas a implementação da especificação, mas também um conjunto abrangente de exemplos e código interoperável.
- O protocolo principal do Matter **enquadra-se nas três camadas superiores do modelo OSI**, o que significa que pode funcionar sobre qualquer tipo de transporte e rede **baseados em IPv6**.
 - Enquanto o controlo e outras comunicações operacionais são realizados sobre IPv6, o *Bluetooth Low Energy* (BLE) pode ser utilizado para integrar (comissionar) novos dispositivos.
- Suporta a **interligação (bridging) com outras tecnologias** de casas inteligentes, como Zigbee, Bluetooth Mesh e Z-Wave.
 - Dispositivos baseados nesses protocolos podem ser operados como se fossem dispositivos Matter através de uma *Bridge*, que é um dispositivo simultaneamente membro da rede Matter e da tecnologia IoT alternativa.
 - As **Bridges acarretam uma dupla vantagem**: os dispositivos que usam outros protocolos passam a ter acesso a tecnologias e ecossistemas concebidos para dispositivos Matter nativos. Simultaneamente, o Matter aproveita tecnologias maduras com uma vasta base instalada de utilizadores.

Tipos de dispositivos suportados

Supported Device Types

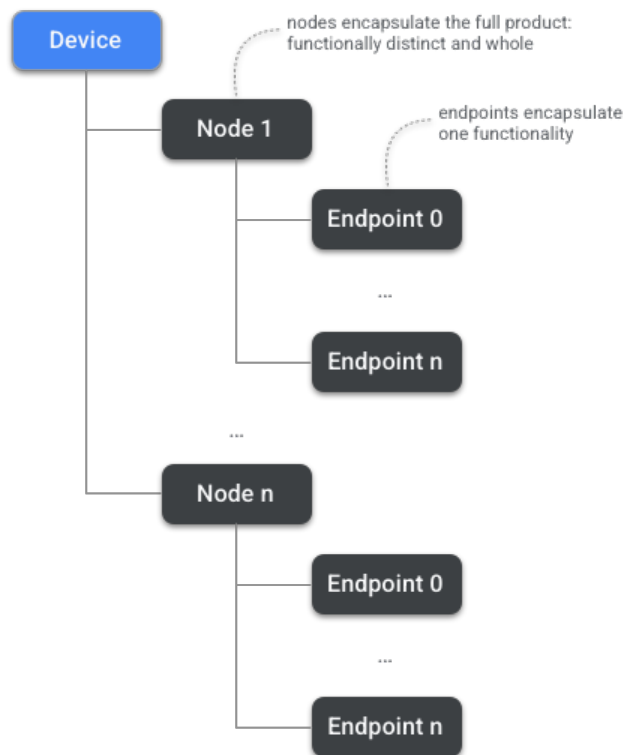
In order to be certified a Matter device must conform to one of the approved device types.

As of Matter 1.4 the following types are certifiable.

- Utility Device Types
 - Root Node
 - Power Source
 - OTA Requestor
 - OTA Provider
 - Bridged Node
 - Electrical Sensor
 - Device Energy Management
 - Application Device Types
 - Lighting
 - On/Off Light
 - Dimmable Light
 - Color Temperature Light
 - Extended Color Light
 - Smart Plugs/Outlets and Other Actuators
 - On/Off Plug-in Unit
 - Dimmable Plug-in Unit
 - Mounted On/Off Control
 - Mounted Dimmable Load Control
 - Pump
 - Water Valve
 - Switches and Controls
 - On/Off Light Switch
 - Dimmer Switch
 - Color Dimmer Switch
 - Control Bridge
 - Pump Controller
 - Generic Switch
- Sensors
 - Contact Sensor
 - Light Sensor
 - Occupancy Sensor
 - Temperature Sensor
 - Pressure Sensor
 - Flow Sensor
 - Humidity Sensor
 - On/Off Sensor
 - Smoke CO Alarm
 - Air Quality Sensor
 - Water Freeze Detector
 - Water Leak Detector
 - Rain Sensor
 - Closures
 - Door Lock
 - Door Lock Controller
 - Window Covering
 - Window Covering Controller
 - HVAC
 - Thermostat
 - Fan
 - Air Purifier
 - Media
 - Basic Video Player
 - Casting Video Player
 - Speaker
 - Content App
 - Casting Video Client
 - Video Remote Control
 - Robotic Devices
 - Robotic Vacuum Cleaner
 - Appliances
 - Refrigerator
 - Temperature Controlled Cabinet
 - Room Air Conditioner
 - Laundry Washer



Modelo de dados de um dispositivo Matter



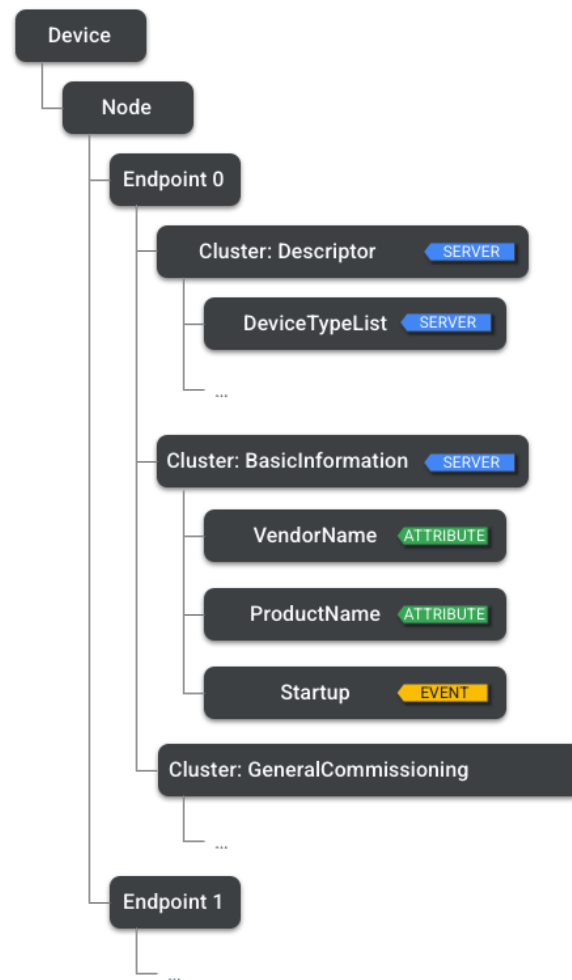
- **Modelo hierárquico** das funcionalidades de um dispositivo com o **Device** na raiz
- Todos os dispositivos são compostos por **Nodes**.
 - Um **Node é um recurso identificável e endereçável** de forma única numa rede, que o utilizador perceciona como uma unidade funcional completa
 - A comunicação no Matter tem origem e destino num *Node*.
 - Os **Nodes são constituídos por um conjunto de Endpoints** onde cada um encapsula um conjunto de funcionalidades específicas
 - Por exemplo, um *Endpoint* pode estar associado à função de iluminação, outro à deteção de movimento e outro à informação básica do *Node* (*vendor ID, product ID, network info*).

Função do *Node*

- A função do *Node* (*Node Role*) corresponde a um conjunto de comportamentos relacionados. Cada *Node* pode ter uma ou mais funções. As funções possíveis incluem:
 - **Commissioner**: *Node* que realiza o processo de *Commissioning* (integração de novos dispositivos na rede).
 - **Controller**: *Node* capaz de controlar um ou mais *Nodes*. Exemplos incluem uma aplicação móvel usada para configurar dispositivos, ou um *Hub* associado a essa aplicação, que pode estar embutido num router, coluna inteligente, televisão ou painel inteligente. Alguns tipos de dispositivos, como o *On/Off Light Switch*, têm o papel de *Controller*.
 - **Controlee**: *Node* que pode ser controlado por um ou mais *Nodes*. A maioria dos tipos de dispositivos podem ser *Controlees*, exceto aqueles cujo papel é de *Controller*, como o *On/Off Light Switch*, que apenas pode ser *Controller* e nunca *Controlee*.
 - **OTA Provider**: *Node* que pode fornecer atualizações de *software Over-The-Air* (OTA).
 - **OTA Requestor**: *Node* que pode solicitar atualizações OTA.



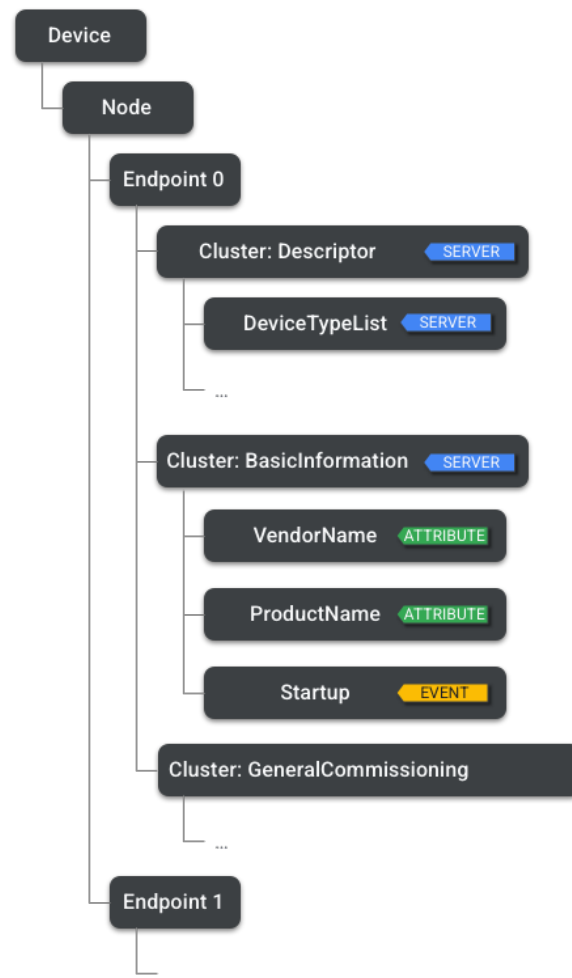
Elementos da Hierarquia de um Node



Clusters

- **Um Endpoint pode conter um ou mais Clusters.**
 - Representam um nível adicional na hierarquia do dispositivo, agrupando funcionalidades específicas: por exemplo, um *on/off cluster* numa tomada inteligente ou um *level control cluster* num ponto de luz regulável (*dimmable light*).
- **Um Node pode também ter vários Endpoints**, cada um representando uma instância independente da mesma funcionalidade.
 - Por exemplo, um candeeiro pode expor o controlo individual de várias luzes, ou uma régua de tomadas pode expor o controlo separado de cada tomada.

Elementos da Hierarquia de um Node



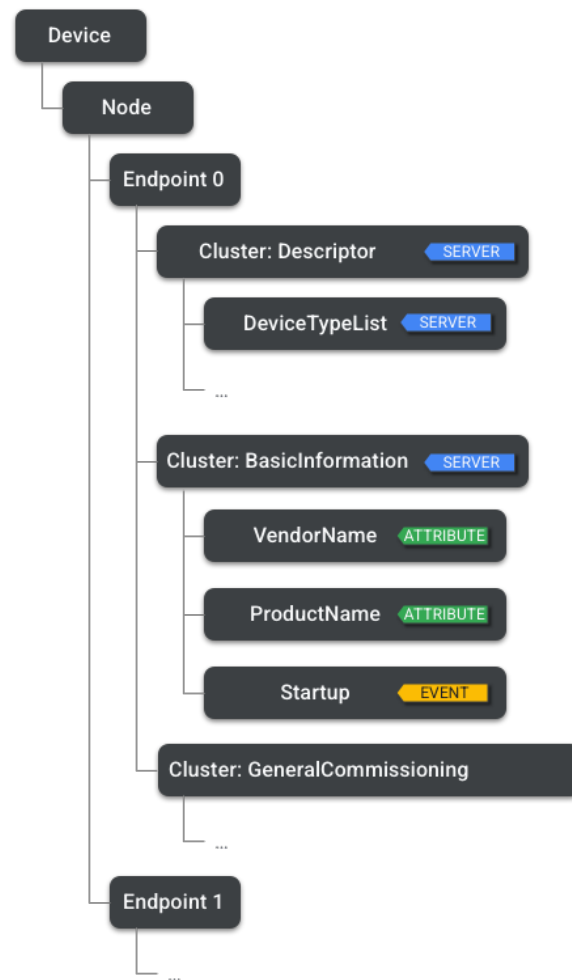
Atributos

- O último nível da hierarquia corresponde aos **Atributos** que representamos estados mantidos pelo *Node*
 - Por exemplo, o atributo de *current level attribute* de um *level control cluster*.
 - Os atributos podem ser definidos com diferentes tipos de dados, como uint8, strings ou arrays.

Comandos

- Representam ações que podem ser executadas.
 - No modelo de dados do Matter, correspondem ao equivalente de uma *remote procedure call*.
 - Possuem natureza verbal, como por exemplo *lock door* num *Door Lock cluster*.
 - Podem gerar respostas e resultados; no Matter, essas respostas também são definidas como comandos, mas no sentido inverso.

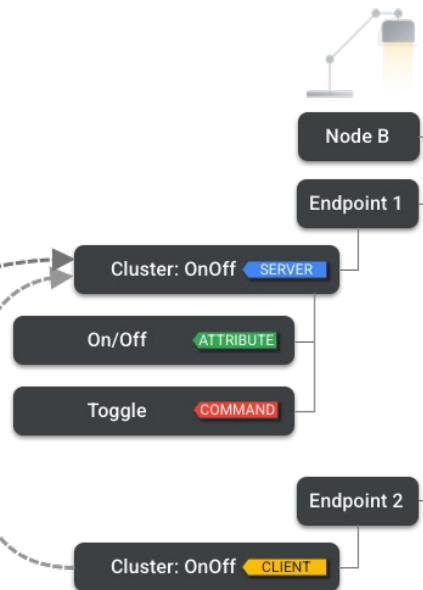
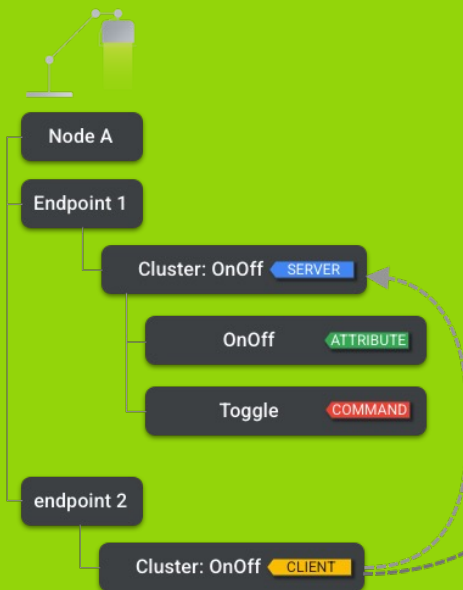
Elementos da Hierarquia de um Node



Eventos

- Os *Clusters* podem conter Eventos, que podem ser entendidos como registros de transições de estado passadas.
 - Enquanto os atributos representam os estados atuais, os eventos funcionam como um diário do passado, contendo um contador monotonicamente crescente, uma marca temporal (*timestamp*) e uma prioridade.
 - Permitem capturar transições de estado e modelar dados que não seriam facilmente representáveis apenas através de atributos.

Clientes e Servidores



Clusters podem ser do tipo *Client* ou *Server*

- **Server** mantém estado e contém Atributos, Eventos e Comandos
- **Client** não tem estado (*stateless*) e tem como responsabilidade iniciar interações com um *Server Cluster* remoto, realizando as seguintes operações:
 - leitura e escrita dos seus Atributos remotos
 - leitura dos seus Eventos remotos
 - invocação dos seus Comandos remotos
- O *On/Off Client Cluster* do **Node A** está a alterar os atributos do *On/Off Server Cluster* tanto do **Node A** como do **Node B**, enquanto o *Client Cluster* do **Node B** está apenas a alterar o *Server Cluster* do próprio **Node B**.

Conectividade Matter

- Em princípio, **qualquer rede que suporte IPv6 é adequada para a implementação do Matter**, desde que suporte um conjunto básico de normas IPv6.
 - Na versão atual da especificação, o foco recai sobre três tecnologias de camada de ligação: Ethernet, Wi-Fi e Thread.
 - A especificação é limitada a estas tecnologias para que o processo de provisionamento possa ser devidamente abrangido e para que o âmbito dos testes de certificação se mantenha controlado
- **Ethernet**: forma de transporte mais simples para o Matter, pois não é necessário transmitir credenciais de rede para o dispositivo; basta ligá-lo à rede Ethernet.
- **Wi-Fi**: dispositivos Matter podem usar o *standard* 802.11 (Wi-Fi) para conectividade. Qualquer uma das versões aprovadas desta especificação é compatível com o Matter.
- **Thread**: protocolo de rádio *mesh* de baixo consumo, baseado em 802.15.4 e concebido para transporte IP.
- **Bluetooth**: o *Bluetooth Low Energy* é utilizado no Matter durante as fases de descoberta e comissionamento, como meio de transmitir as credenciais da rede Wi-Fi ou Thread que o dispositivo usará para a comunicação operacional.



Modelo de interação: Leitura

Read Request Action



Read Request Action (Iniciador → Alvo)

- O iniciador envia um pedido de leitura ao alvo. Pode incluir:
 - *Attribute Requests*: lista de caminhos para os atributos a ler.
 - *Event Requests*: lista de caminhos para os eventos a ler.
- Após receber o pedido, o alvo prepara uma *Report Data Action* com a informação solicitada.

Report Data Action (Alvo → Iniciador)

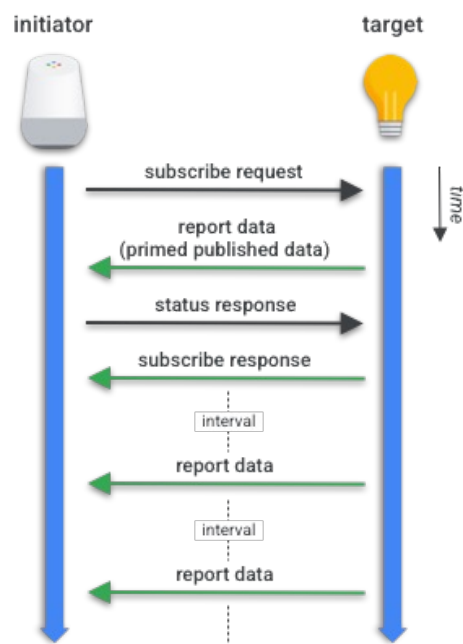
- O alvo responde com:
 - *Attribute Reports*: valores dos atributos pedidos.
 - *Event Reports*: eventos pedidos.
 - *Suppress Response*: indica se deve suprimir a resposta de estado.
 - *Subscription ID*: identifica uma transação de subscrição, se aplicável.

Status Response Action (Iniciador ↔ Alvo)

- Por defeito, o iniciador envia esta ação para confirmar a receção dos dados.
- Se o *Suppress Response* estiver ativo, não é enviada.
- Após este passo, a operação de leitura está concluída.

Modelo de interação: Leitura

Subscribe Request Action



Subscribe Request Action (Iniciador → Alvo)

- O iniciador (*Subscriber*) solicita ao alvo (*Publisher*) atualizações periódicas de atributos ou eventos. O pedido inclui:
 - Min Interval Floor* – intervalo mínimo entre relatórios.
 - Max Interval Ceiling* – intervalo máximo entre relatórios.
 - Listas de atributos e eventos a monitorizar.
- O alvo responde com uma *Report Data Action* inicial contendo os primeiros dados (*Primed Published Data*).

Status Response Action (Iniciador → Alvo)

- O iniciador confirma a receção dos dados iniciais sem erros.

Subscribe Response Action (Alvo → Iniciador)

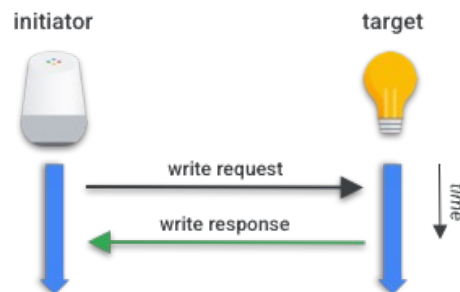
- Conclui o processo de subscrição, enviando:
 - Subscription ID* – identificador da subscrição.
 - Min Interval* e *Max Interval* finais acordados.

Ciclo de Subscrição

- Após estabelecida, o alvo envia periodicamente *Report Data Actions* com novas atualizações.
- O iniciador responde a cada relatório até que a subscrição seja perdida ou cancelada.

Modelo de interação: Escrita

Untimed Write Transaction



Write Request Action (Iniciador → Alvo)

- O iniciador envia ao alvo dados a gravar, incluindo:
 - *Write Requests*: lista de pares (*Path*, dados).
 - *Timed Request*: indica se faz parte de uma transação temporizada (*Timed Write*).
 - *Suppress Response*: indica se a resposta deve ser suprimida.

Write Response Action (Alvo → Iniciador)

- Após receber o pedido, o alvo responde com uma lista de *Write Responses*, indicando para cada caminho o resultado ou código de erro da operação.

No caso da **Timed Write Transaction**:

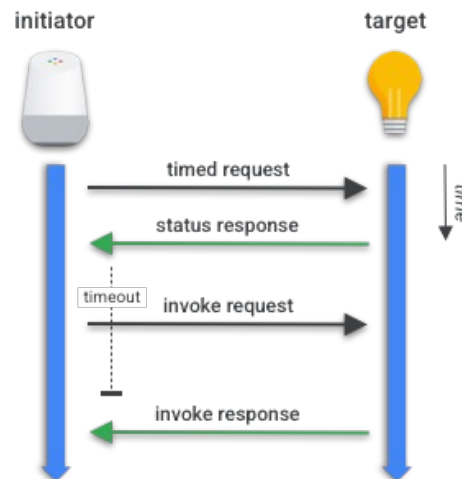
- O Iniciador inicia a transação enviando uma *Timed Request Action* que define um tempo limite (*timeout*) em milissegundos durante o qual a transação permanece válida.
- O Alvo responde com uma *Status Response Action* a confirmar a receção sem erros.
- Depois dessa confirmação, o Iniciador envia a *Write Request Action* dentro do período definido, repetindo-se o processo acima.

As ações **Timed Request**, **Write Request** e **Write Response** são exclusivamente **unicast** (isto é, comunicam apenas entre dois nós específicos).



Modelo de interação: Escrita

Timed Invoke Transaction



Invoke Request Action (Iniciador → Alvo)

- O iniciador envia um ou mais comandos de *cluster* ao alvo, incluindo:
 - *Invoke Requests*: caminhos dos comandos e eventuais parâmetros (Command Fields).
 - *Timed Request*: indica se faz parte de uma transação temporizada.
 - *Suppress Response*: indica se deve suprimir a resposta.
 - *Interaction ID*: identifica a interação entre pedido e resposta.

Invoke Response Action (Alvo → Iniciador)

- O alvo responde com:
 - **Invoke Responses**: resultados ou estados de cada comando invocado.
 - O mesmo *Interaction ID* para associar resposta e pedido.

No caso da Timed Invoke Transaction:

- O Iniciador envia uma *Timed Request Action* com um tempo limite (*timeout*) em milissegundos, durante o qual a transação permanece válida.
- O Alvo responde com uma *Status Response Action* a confirmar a receção sem erros.
- Após essa confirmação, o Iniciador envia a *Invoke Request Action* dentro do período definido, repetindo-se o processo identificado.

Constrained Application Protocol

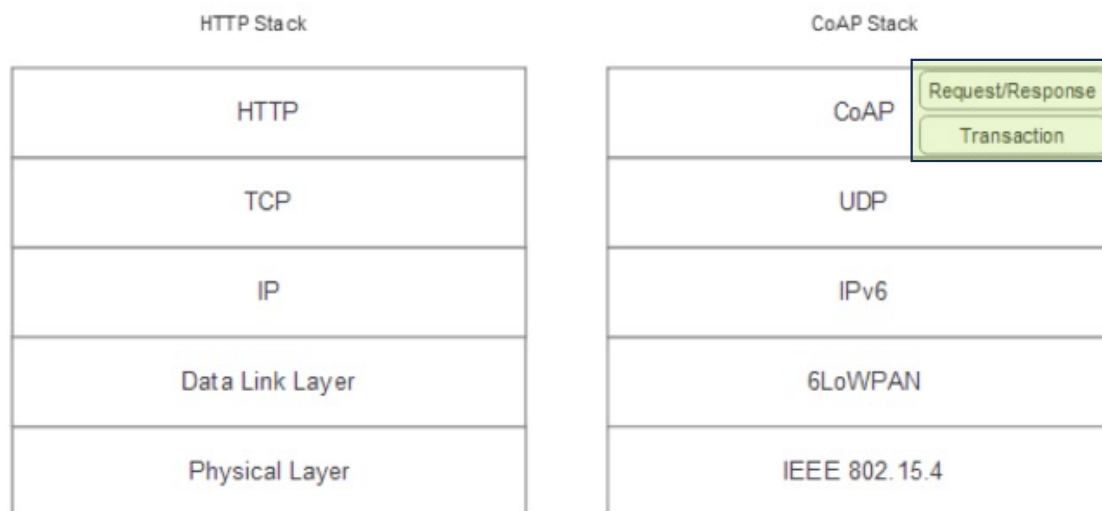
- O **Constrained Application Protocol (CoAP)** foi desenvolvido pelo IETF, no âmbito do grupo CoRE (*Constrained RESTful Environments*), com a primeira proposta publicada em 2014.
- É um protocolo de **comunicação para dispositivos com recursos limitados**, definido formalmente na norma **RFC7252**.
- Foi concebido para **comunicação máquina-a-máquina** (M2M) entre nós periféricos (*edge nodes*).
- Suporta mapeamento para HTTP através de proxies, permitindo a troca de dados pela Internet.
- Oferece uma **estrutura de endereçamento de recursos semelhante à da Web**, mas mais leve em termos de consumo de largura de banda e memória.
- Estudos demonstram que o **CoAP é mais eficiente que o HTTP**, oferecendo funcionalidades equivalentes com menor overhead e menor consumo de energia.

Constrained Application Protocol

- O CoAP **baseia-se no conceito de imitar e substituir as funcionalidades pesadas do HTTP** por um equivalente leve, adequado ao IoT.
- **Não é um substituto direto do HTTP**, pois carece de algumas funcionalidades - o HTTP destina-se a sistemas mais exigentes e orientados a serviços.
- As **principais características do CoAP** podem ser resumidas da seguinte forma:
 - Estrutura semelhante ao HTTP
 - Baseado em **protocolos sem ligação** (connection-less)
 - **Segurança através de DTLS**, em vez de TLS usado nas transmissões HTTP normais
 - Trocas de mensagens assíncronas
 - **Design leve, com baixo consumo de recursos e cabeçalhos reduzidos**
 - Suporte para URI e tipos de conteúdo (content-types)
 - **Construído sobre UDP**, em contraste com o TCP/UDP usado no HTTP tradicional
 - Mapeamento HTTP sem estado, permitindo o uso de proxies para fazer a ponte com sessões HTTP



Camadas protocolares do CoAP



- **Camada de Pedido/Resposta:** responsável por enviar e receber consultas baseadas em REST. As consultas REST são transportadas (*piggybacked*) em mensagens CON ou NON, e as respostas REST são transportadas na respetiva mensagem ACK correspondente.
- **Camada Transaccional:** gere trocas individuais de mensagens entre pontos finais, utilizando um dos quatro tipos básicos de mensagem. Esta camada também oferece suporte a *multicast* e controlo de congestionamento.

Características do protocolo CoAP

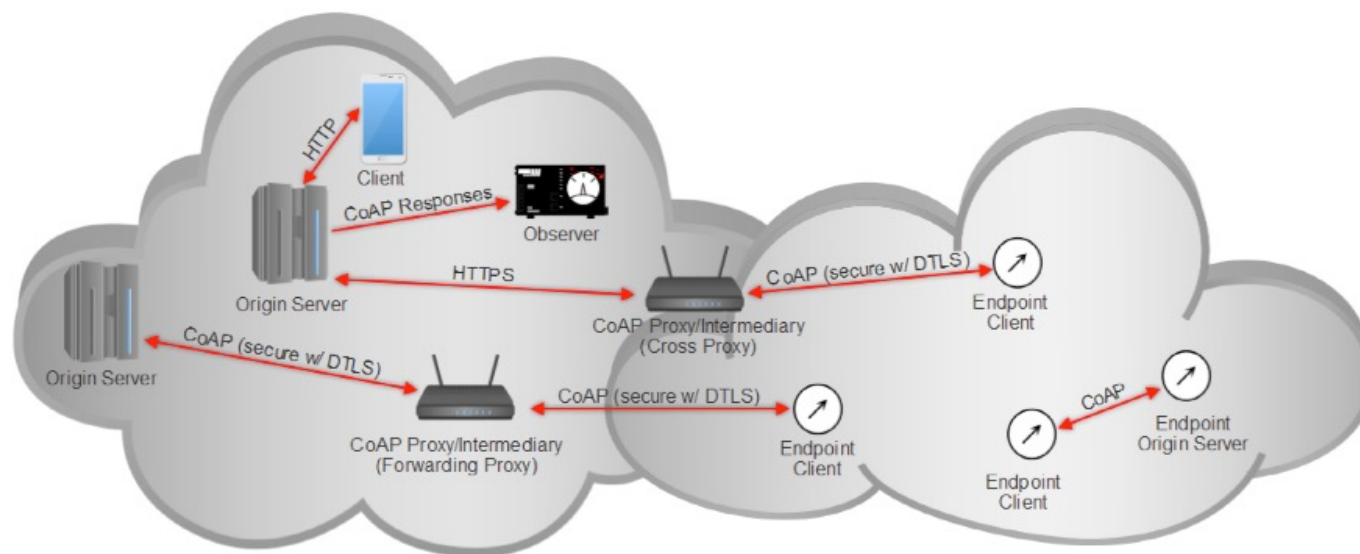
- O **CoAP** partilha o mesmo contexto, sintaxe e forma de utilização do HTTP. O endereçamento em CoAP segue igualmente o estilo do HTTP, estendendo-se à estrutura URI.
- Tal como nos URIs do HTTP, o **utilizador deve conhecer previamente o endereço** para aceder a um recurso.
- No nível mais alto, o CoAP utiliza **os mesmos tipos de pedidos que o HTTP**: GET, PUT, POST e DELETE.
- De forma semelhante, os códigos de resposta imitam os do HTTP, por exemplo:
 - 2.01: *Created*
 - 2.02: *Deleted*
 - 2.04: *Changed*
 - 2.05: *Content*
 - 4.04: *Not Found* (recurso)
 - 4.05: *Method Not Allowed*
- A forma típica de um URI em CoAP é:

coap://host[:port]/[path][?query]

Atores do sistema CoAP

- **Endpoints:** são as origens e destinos das mensagens CoAP. A definição exata de um *endpoint* depende do tipo de transporte utilizado.
- **Proxies:** um *endpoint* CoAP encarregado de executar pedidos em nome dos clientes CoAP. As suas funções incluem reduzir o tráfego de rede, aceder a nós em modo de suspensão e fornecer uma camada adicional de segurança.
- Os proxies podem ser:
 - **Forward-proxying:** selecionados explicitamente por um cliente para encaminhar pedidos.
 - **Reverse-proxying:** atuam como servidores locais (*in-situ*).
 - **Cross-proxying:** traduzem entre diferentes protocolos, por exemplo, de CoAP para HTTP. Um caso comum é um *router* de fronteira que faz *proxy* entre uma rede CoAP e serviços HTTP na nuvem.
- **Client:** origem de um pedido e destino da resposta.
- **Server:** destino de um pedido e origem da resposta.
- **Intermediary:** cliente que atua simultaneamente como servidor e cliente em relação a um servidor de origem. Um *proxy* é um tipo de *intermediary*.
- **Origin servers:** servidores onde o recurso solicitado está efetivamente localizado.
- **Observers:** clientes que se registam como observadores através de uma mensagem GET modificada. O servidor mantém a ligação e envia notificações ao observador sempre que o estado do recurso se altera.

Arquitectura CoAP



- Por ser um sistema HTTP leve, o CoAP **permite que clientes comuniquem entre si ou com serviços na nuvem que suportem CoAP**.
- Pode-se usar um **proxy para fazer a ponte com serviços HTTP** baseados na nuvem.
- Os **endpoints CoAP** podem estabelecer relações diretas entre si, inclusive ao nível dos sensores.
- O **mecanismo de Observers** permite subscrever recursos cujos estados mudam, de forma semelhante ao funcionamento do MQTT.
- Os servidores de origem (*origin servers*) armazenam os recursos partilhados.
- Os *proxies* desempenham dois papéis principais:
 - Traduzir entre CoAP e HTTP (*HTTP translation*).
 - Reencaminhar pedidos em nome de um cliente (*forwarding requests*).

Formatos de Mensagem CoAP

Confirmável (CON)

- Requer um ACK.
- Quando uma mensagem CON é enviada, o ACK deve ser recebido dentro de um intervalo aleatório entre ACK_TIMEOUT e $(ACK_TIMEOUT \times ACK_RANDOM_FACTOR)$.
- Se o ACK não for recebido, o emissor reitera a transmissão da mensagem CON em intervalos exponencialmente crescentes até receber o ACK ou um RST.
- Este é o mecanismo de controlo de congestionamento do CoAP.
- Existe um número máximo de tentativas definido por MAX_RETRANSMIT.
- Este processo fornece resiliência adicional, compensando a falta de fiabilidade do UDP.

Não confirmável (NON)

- Não requer ACK.
- Corresponde a uma mensagem do tipo “*fire-and-forget*”, ou seja, enviada sem esperar resposta, podendo ser usada para difusão (*broadcast*).

Confirmação (ACK)

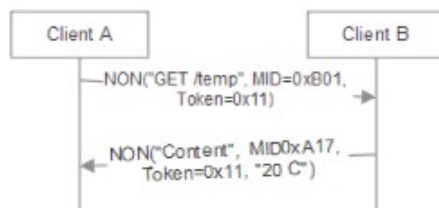
- Confirma a receção de uma mensagem CON.
- Pode transportar (*piggyback*) outros dados juntamente com o ACK.

Reinicialização (RST)

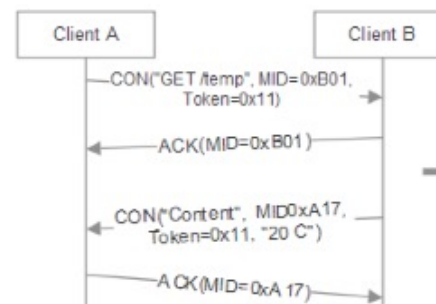
- Indica que uma mensagem CON foi recebida, mas o contexto está em falta.
- Tal como o ACK, a mensagem RST também pode incluir outros dados.

Formatos de Mensagem CoAP

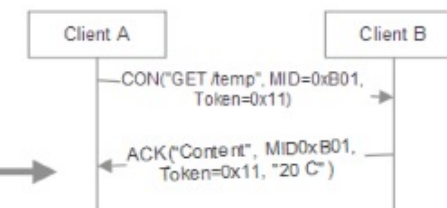
CoAP Example: Non-confirmable request and response



CoAP Example: Confirmable request and response



CoAP Example: Confirmable with Piggyback (alternate)



- **Pedido/resposta não confirmável (à esquerda):** Uma mensagem enviada entre os clientes A e B utilizando a estrutura típica de um HTTP GET. O cliente B responde posteriormente com dados de conteúdo, devolvendo por exemplo a temperatura de 20 °C.
- **Pedido/resposta confirmável (ao centro):** Inclui o Message-ID, identificador único de cada mensagem. O *Token* representa um valor que deve coincidir durante toda a troca de mensagens.
- **Pedido/resposta confirmável (à direita):** A mensagem é confirmável. Tanto o cliente A como o cliente B aguardam um ACK após cada troca de mensagens. Para otimizar a comunicação, o cliente B pode anexar (*piggyback*) o ACK juntamente com os dados de resposta, como ilustrado à direita.

Kahoot! Time

